

C'est une excellente question ! Les annotations sont partout en Java. Pour bien comprendre, on va faire ça en deux étapes : d'abord avec une image simple, puis avec les vrais termes techniques.

Partie 1 : L'explication pour un enfant

Imagine que tu as une boîte de jouets. Pour savoir ce qu'il y a dedans sans l'ouvrir, tu colles une **étiquette** dessus avec un dessin de voiture, ou un mot écrit "Lego".

L'étiquette en elle-même n'est pas un jouet. Elle ne fait rien du tout. Mais elle donne une **information** à toi, ou à tes parents qui rangent la chambre.

En Java, une **annotation**, c'est exactement ça : une **étiquette magique** que l'on colle sur notre code (sur une boîte, une fonction, ou une variable).

- Le code lui-même ne change pas.
 - Mais le "robot" qui lit le code (l'ordinateur) voit l'étiquette et sait qu'il doit faire quelque chose de spécial, ou faire attention.
-

Partie 2 : L'explication normale (Développeur)

En Java, les **annotations** sont des **métadonnées** (des données sur les données) que l'on ajoute au code source. Elles n'affectent pas directement le fonctionnement logique du code, mais elles fournissent des instructions :

1. **Au compilateur** (ex: pour détecter des erreurs ou masquer des avertissements).
2. **Aux outils de build / IDE** (ex: pour générer du code ou de la documentation).
3. **Au moment de l'exécution (Runtime)** (ex: pour configurer des frameworks comme Spring ou Hibernate via la réflexion).

À quoi ça sert ?

- **Éviter la configuration XML lourde** : Avant, on configurait tout dans des fichiers texte séparés. Maintenant, on met une annotation directement sur la classe.
- **Générer du code automatiquement** : Par exemple, créer des getters/setters automatiquement (comme avec la bibliothèque Lombok).
- **Guider le compilateur** : Lui dire "je sais que cette méthode remplace une autre" (**@Override**).

Les annotations de base les plus connues

- **@Override** : Indique qu'une méthode redéfinit une méthode de sa classe parente.
 - **@Deprecated** : Signale que la méthode est ancienne et ne doit plus être utilisée.
 - **@SuppressWarnings** : Dit au compilateur "Ne m'affiche pas d'avertissement ici, je sais ce que je fais".
-

Partie 3 : Comment créer une annotation personnalisée

Pour créer ta propre "étiquette", tu utilises le mot-clé `@interface`. On utilise aussi des **méta-annotations** (des annotations pour annoter notre annotation) pour dire comment elle doit se comporter.

Voici un exemple concret : on va créer une annotation `@Hero` pour marquer certaines classes et leur donner un niveau de puissance.

1. Définition de l'annotation

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

// 1. @Target dit OÙ on peut coller l'étiquette (ici, sur une CLASSE / TYPE)
@Target(ElementType.TYPE)

// 2. @Retention dit JUSQU'À QUAND l'étiquette reste (Runtime = visible pendant que le programme tourne)
@Retention(RetentionPolicy.RUNTIME)
public @interface Hero {
    String superPouvoir() default "Voler"; // Un paramètre avec une valeur par défaut
    int niveauPuissance();                // Un paramètre obligatoire
}
```

2. Utilisation de l'annotation

Maintenant, on colle notre étiquette sur une classe `Guerrier`.

```
@Hero(superPouvoir = "Super Force", niveauPuissance = 90)
public class Guerrier {
    String nom = "Thor";
}
```

3. Comment on l'utilise dans le programme (La Réflexion)

Pour que l'annotation serve à quelque chose, on va écrire un code qui "lit" cette étiquette au moment où le programme s'exécute.

```
public class Main {
    public static void main(String[] args) {
        Guerrier monPerso = new Guerrier();
    }
}
```

```

        // On inspecte la classe de l'objet pour voir s'il a l'étiquette
        @Hero
        if (monPerso.getClass().isAnnotationPresent(Hero.class)) {

            // On récupère l'étiquette
            Hero annotationHero =
monPerso.getClass().getAnnotation(Hero.class);

            // On lit ce qui est écrit dessus
            System.out.println("Ce personnage est un Héros !");
            System.out.println("Pouvoir : " +
annotationHero.superPouvoir());
            System.out.println("Puissance : " +
annotationHero.niveauPuissance());
        } else {
            System.out.println("C'est un personnage normal.");
        }
    }
}

```

Résultat dans la console :

```

Ce personnage est un Héros !
Pouvoir : Super Force
Puissance : 90

```

En résumé, l'annotation a permis d'injecter des informations ("Super Force", 90) directement sur la classe, et notre programme a pu les lire pour adapter son comportement !

—
PROF

Quand l'utiliser??

Maintenant que tu as compris le concept, la vraie question en tant que développeur, c'est : **quand est-ce que j'ai intérêt à créer ou utiliser une annotation dans mon code ?**

On utilise les annotations principalement dans quatre grands scénarios :

1. Pour automatiser et générer du code répétitif (Boilerplate)

Si tu en as marre d'écrire des getters, des setters, des constructeurs ou des méthodes `toString()`, les annotations sont tes meilleures amies.

- **L'exemple roi** : La bibliothèque **Lombok**. En posant juste `@Data` sur une classe, elle génère tout ce code fastidieux en arrière-plan pendant la compilation. Ton code reste propre et court.

2. Pour configurer des Frameworks sans fichiers XML

C'est l'utilisation la plus courante en entreprise (avec Spring, Hibernate, ou Jakarta EE). Au lieu d'écrire un gros fichier de configuration à côté, tu indiques le comportement directement là où se trouve le code.

- **Pour le Web (Spring) :** Tu mets `@RestController` sur une classe pour dire "Ceci est un point d'entrée pour mon API Web", et `@GetMapping("/users")` sur une méthode pour dire "Exécute cette fonction quand quelqu'un visite l'URL `/users`".
- **Pour la Base de données (JPA/Hibernate) :** Tu mets `@Entity` sur une classe et `@Table(name = "utilisateurs")` pour l'associer automatiquement à une table SQL.

3. Pour appliquer des règles de validation de données

Au lieu d'écrire des dizaines de conditions `if (email == null || !email.contains("@"))`, tu peux utiliser des annotations de validation standard (Jakarta Bean Validation).

- Tu mets `@NotBlank` ou `@Email` ou `@Min(18)` directement au-dessus des variables de ton modèle. Le framework bloquera automatiquement les données incorrectes avant même qu'elles ne traitent ta logique métier.

4. Pour créer des "Intercepteurs" ou des comportements transversaux (AOP)

C'est le cas parfait pour créer tes propres annotations personnalisées. Imagine que tu veuilles mesurer le temps d'exécution de plusieurs fonctions, ou vérifier si l'utilisateur est connecté avant d'exécuter une méthode.

Au lieu de copier-coller le code de vérification partout, tu crées une annotation (par exemple `@CheckAspect` ou `@LogExecutionTime`). Grâce à la Programmation Orientée Aspect (AOP), tu peux dire à ton programme : *"À chaque fois que tu croises cette étiquette, exécute ce code de sécurité d'abord"*.

💡 **Résumé :** La règle d'or pour savoir quand l'utiliser

PROF

N'en abuse pas pour du code purement local. Si une logique ne concerne qu'une seule fonction de ton application, écris du code Java classique.

Utilise les annotations quand : Tu veux extraire une configuration, automatiser une tâche répétitive, ou appliquer un même comportement à plein d'endroits différents dans ton application de manière transparente.

Comment savoir si c'est pour une méthode, classe ou attribut?

Dans la définition (le fichier source de l'annotation elle-même), pour savoir à quoi elle sert, c'est **uniquement** la méta-annotation `@Target` qui te le dit.

C'est une ligne obligatoire (ou presque) placée tout en haut, juste au-dessus du mot-clé `@interface`.

Voici les trois codes de définition pour que tu voies la différence exacte :

1. Si elle est faite pour une CLASSE

Tu verras la valeur **ElementType.TYPE** :

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Target;

@Target(ElementType.TYPE) // <-- C'est ça qui dit "C'est pour une classe
/ interface"
public @interface PourClasseUniquement {
}
```

2. Si elle est faite pour une MÉTHODE

Tu verras la valeur **ElementType.METHOD** :

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Target;

@Target(ElementType.METHOD) // <-- C'est ça qui dit "C'est pour une
méthode / fonction"
public @interface PourMethodeUniquement {
}
```

3. Si elle est faite pour un ATTRIBUT

Tu verras la valeur **ElementType.FIELD** :

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Target;

@Target(ElementType.FIELD) // <-- C'est ça qui dit "C'est pour un
attribut / variable de classe"
public @interface PourAttributUniquement {
}
```

 Et si elle fait plusieurs choses à la fois ?

Si le créateur de l'annotation a voulu qu'elle serve à la fois pour les méthodes et les attributs, il liste les cibles entre accolades `{}` séparées par une virgule :

```
@Target({ElementType.METHOD, ElementType.FIELD}) // <-- Pour les deux !
public @interface PourMethodeOuAttribut {
}
```

En résumé : Dans le fichier de définition, tu ignores tout le reste et tu regardes uniquement le mot dans les parenthèses de `@Target(ElementType....)`. C'est l'unique indicateur officiel.

TP

Voici un exercice pratique à réaliser directement dans ton IDE. Le but est de créer un mini-système de sécurité basé sur des rôles (comme dans un vrai site web) en utilisant une annotation personnalisée.

L'énoncé : Le Système de Sécurité "VIP"

Tu dois développer un système qui restreint l'accès à certaines méthodes d'une classe en fonction du rôle de l'utilisateur (par exemple : `ADMIN`, `USER`, ou `GUEST`).

Si un utilisateur n'a pas le bon rôle, le programme doit bloquer l'exécution et afficher un message d'erreur.

Tes missions (Étape par étape)

Étape 1 : Créer l'annotation `@Autorisation`

Crée une annotation personnalisée nommée `Autorisation`.

- Elle doit être applicable **uniquement sur les méthodes** (`@Target`).
- Elle doit être visible **au moment de l'exécution** (`@Retention`).
- Elle doit accepter un paramètre de type `String` nommé `roleRequis`.

Étape 2 : Créer la classe `ServiceFichiers`

Crée une classe avec deux méthodes :

1. `lireFichier()` : accessible par tout le monde (pas besoin d'annotation).
2. `supprimerFichier()` : colle l'annotation dessus avec le rôle requis `"ADMIN"`.

Étape 3 : Créer le moteur de sécurité (La Réflexion)

C'est ici que la magie opère. Tu vas créer une classe `ContrôleurSecurite` avec une méthode statique :

```
public static void executerMethode(Object objet, String nomMethode,
String roleUtilisateur Actuel)
```

Dans cette méthode, tu dois :

1. Récupérer la méthode de l'objet grâce à son nom (en utilisant `objet.getClass().getMethod(nomMethode)`).
2. Vérifier si cette méthode possède l'annotation `@Autorisation`.
3. **Si oui** : Comparer le `roleUtilisateurActuel` avec le `roleRequis` de l'annotation. Si ça correspond, tu affiches "Accès autorisé". Sinon, tu affiches "Accès refusé ! Rôle ADMIN requis."
4. **Si non** : (La méthode n'a pas d'annotation), tu affiches "Accès autorisé (Public)".

Le code de départ pour tester (Main)

Une fois que tu as écrit tes classes, utilise ce code dans ton `main` pour vérifier si ton système fonctionne :

```
public class Main {
    public static void main(String[] args) throws Exception {
        ServiceFichiers monService = new ServiceFichiers();

        System.out.println("--- TEST 1 : Un GUEST essaie de lire ---");
        ControleurSecurite.executerMethode(monService, "lireFichier",
"GUEST");
        // Attendu : Accès autorisé (Public)

        System.out.println("\n--- TEST 2 : Un GUEST essaie de supprimer
---");
        ControleurSecurite.executerMethode(monService,
"supprimerFichier", "GUEST");
        // Attendu : Accès refusé ! Rôle ADMIN requis.

        System.out.println("\n--- TEST 3 : Un ADMIN essaie de supprimer
---");
        ControleurSecurite.executerMethode(monService,
"supprimerFichier", "ADMIN");
        // Attendu : Accès autorisé
    }
}
```

PROF

À toi de jouer ! Prends ton temps pour manipuler l'API de réflexion (`java.lang.reflect`). Dis-moi quand tu as terminé ou si tu bloques sur l'écriture du `ControleurSecurite`, et je te donnerai la correction détaillée.